CENTRO UNIVERSITÁRIO INTERNACIONAL UNINTER

CURSO: CST Análise e desenvolvimento de sistemas

JEFERSON RIVALDO SANTOS

RU: 4937546

RAÍZES DO NORDESTE

PROJETO: DESENVOLVIMENTO BACK-END (898968)

Polo de apoio: A INFORMAR

Semestre/período: B fase II 2026 - regular

Professor(a): Luciane Yanase Kanashiro

A INFORMAR

2026

JEFERSON RIVALDO SANTOS

RU: 4937546

RAÍZES DO NORDESTE

PROJETO: DESENVOLVIMENTO BACK-END (898968)

Trabalho apresentado à disciplina Projeto: Desenvolvimento Back-end (898968), do curso CST Análise e desenvolvimento de sistemas, no Centro Universitário Internacional UNINTER. Desenvolvimento de uma API REST em Java para a rede Raízes do Nordeste. Professor(a): Luciane Yanase Kanashiro.

A INFORMAR

2026

SUMÁRIO

1. Introdução e objetivos 3

2. Análise e requisitos 4

3. Modelagem e arquitetura 6

4. API e contratos 9

5. LGPD, privacidade, segurança e auditoria 10

6. Plano de testes e evidências 12

7. Entrega técnica 13

8. Conclusão 14

REFERÊNCIAS 15

APÊNDICE A — CONTRATOS E EXEMPLOS DA API 16

APÊNDICE B — CENÁRIOS DE TESTE DA API 35

APÊNDICE C — EVIDÊNCIAS DE EXECUÇÃO 39

APÊNDICE D — DIAGRAMA DE CASOS DE USO 41

APÊNDICE E — DIAGRAMA CONCEITUAL DE CLASSES 42

APÊNDICE F — ARQUITETURA EM CAMADAS 43

APÊNDICE G — DIAGRAMA ENTIDADE-RELACIONAMENTO 44

1. Introdução e objetivos

A rede de lanchonetes precisa integrar unidades com cardápio e estoque próprios, atendendo APP, TOTEM, BALCAO, PICKUP e WEB. Esta implementação acadêmica oferece uma API REST persistente que centraliza pedidos, pagamento simulado, operação da cozinha e fidelização. O canal de origem é obrigatório e consultável, permitindo acompanhamento operacional. O objetivo é fechar o fluxo pedido → pagamento mock → preparo → entrega, com validação de estoque e controle de acesso.

Este relatório considera o Estudo de Caso e a Trilha Back-End da rede Raízes do Nordeste (UNINTER, 2026), nas versões editadas para estudo fornecidas pelo estudante. A solução prioriza o MVP executável e a documentação das decisões. Os atores são cliente, atendente, cozinha, gerente, administrador e gateway de pagamento simulado. Antes da entrega institucional, as orientações devem ser conferidas nos documentos oficiais completos e no manual acadêmico disponível no AVA.

2. Análise e requisitos

Tabela 1 — ID e informações relacionadas

ID	Requisito	Prioridade e evidência
RF01	Cadastro e autenticação por token; autorização por perfil	Essencial; /usuarios, /auth/login; hash BCrypt e JWT
RF02	Gestão da rede/unidades	Essencial; cadastro/atualização ADMIN; gerente atualiza sua unidade
RF03	Catálogo global e cardápio por unidade	Essencial; produtos, preços em centavos e disponibilidade
RF04	Criação e consulta de pedidos com itens e canalPedido	Essencial; canal obrigatório e filtro por canal/status
RF05	Atualização de status e cancelamento	Essencial; máquina de estados com autorização operacional
RF06	Estoque por unidade e movimentos	Essencial; operações atômicas e proibição de saldo negativo
RF07	Pagamento externo simulado e registro de retorno	Essencial; gateway desacoplado, aprovação, negativa, erro e estorno mock
RF08	Fidelidade com consentimento e resgate	Implementado; 1 ponto por real efetivamente pago na entrega; resgate de 100 pontos = R$ 10
RF09	Promoção/campanha	Implementado; cupom NORDESTE10: 10% sobre subtotal mínimo de R$ 50, sem vigência dinâmica
RF10	Privacidade e auditoria	Implementado; leitura de dados próprios, retirada de consentimento e desidentificação
RF11	Indicadores consolidados da matriz	Proposto; vendas por unidade/região, produtos mais vendidos e relatórios financeiros, com acesso restrito à gestão
RF12	Variações e sazonalidade do cardápio	Parcial; disponibilidade e preços locais implementados; vigência sazonal e receitas regionais propostas
RF13	Campanhas segmentadas	Proposto; segmentação por frequência com consentimento específico; coleta de idade não integra o MVP
RF14	Regras de funcionamento por unidade	Proposto; horários, capacidade da cozinha e metas operacionais demandam modelagem adicional

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Tabela 2 — ID e informações relacionadas

ID	Requisito não funcional	Decisão e limite
RNF01	Segurança	JWT HS256 com expiração, hash BCrypt, perfis e vínculo com unidade; HTTPS deve ser configurado no deploy
RNF02	Integridade	FK/CHECK/UNIQUE e transação Spring JDBC com bloqueios FOR UPDATE; estoque, pontos e pedido confirmam juntos
RNF03	Rastreabilidade	Auditoria de operações sensíveis e leituras; requestId por resposta; sem senha/token nos registros
RNF04	Documentação	Swagger /docs, OpenAPI exportado e contratos de entrada/saída tipados

ID	Requisito não funcional	Decisão e limite
RNF05	Desempenho	Paginação até 100 registros e índices de busca; testes de concorrência; sem alegar benchmark de produção
RNF06	Disponibilidade	Healthcheck com consulta real ao banco; H2 local não oferece alta disponibilidade
RNF07	Tolerância a falhas	Mock indisponível gera registro ERRO; nova chave permite nova tentativa; não autoriza cozinha antes de aprovação
RNF08	Reprodutibilidade	Java 21, versões Maven fixadas e checksums estritos, migrations versionadas, seed idempotente e testes isolados

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Escolheu-se H2 para dispensar um serviço externo no ambiente de estudo. Locks transacionais por recurso protegem estoque e fidelidade; essa decisão limita a concorrência de uma rede em produção. Uma evolução real exige PostgreSQL, controle de concorrência, filas, backups, monitoramento e medição de carga. A integração mock foi escolhida por exigência do roteiro e não coleta dados de cartão.

A priorização atende primeiro à venda consistente e rastreável, evitando estoque negativo e pagamento duplicado. Os indicadores gerenciais, a sazonalidade e as campanhas segmentadas são relevantes ao caso, mas ficam como evolução explicitamente documentada. Alta disponibilidade e desempenho em pico são requisitos de produção; o projeto apresenta mecanismos básicos e uma proposta de evolução, sem afirmar validação de carga ou disponibilidade contínua.

3. Modelagem e arquitetura

Os diagramas estão em docs/diagramas/: casos de uso, DER completo, classes conceituais e arquitetura em SVG/PNG, com fontes DOT. A sequência do fluxo crítico está também em docs/sequencia.md.

A API valida DTOs Java com Bean Validation e aplica autenticação. Application orquestra transações e gateway. Domain define enums, transições, preços e regras de resgate/cupom. Infrastructure contém H2, migrations Flyway SQL, hashing, JWT e gateway simulado. O domínio é implementado com funções e registros persistentes; o diagrama de classes é uma visão conceitual dessas entidades, não uma alegação de ORM ou DDD completo.

Uma unidade possui vários produtos em estoque, e cada produto pode estar em várias unidades; stocks usa PK composta (unidade_id, produto_id). Usuário possui pedidos, movimentos de fidelidade e auditoria. Pedido pertence a uma unidade e possui itens e tentativas de pagamento independentes. Itens guardam o nome/preço histórico do produto. Chaves de idempotência são únicas por ator e operação. A migration V1__initial.sql define todas as tabelas, PK, FK, CHECK e índices.

Tabela 3 — Tabela e informações relacionadas

Tabela	Chaves e relacionamento	Integridade relevante
units / products	PK id; cadastros da unidade e do catálogo	Desativação preserva o histórico
users	PK id; FK unidade_id opcional para equipe	Email único; perfil limitado; pontos não negativos
stocks	PK composta unidade_id/produto_id; FK para units/products	Quantidade não negativa; preço positivo
orders	PK id; FK cliente_id/unidade_id	Canal e status limitados por CHECK; total não negativo
order_items	PK id; FK pedido_id/produto_id	Produto único por pedido; quantidade e preço positivos
payments	PK id; FK pedido_id	Várias tentativas por pedido; status e payload persistidos
stock_movements	PK id; FK composta para stocks; FK ator e pedido	Movimentação vinculada ao recurso e ao responsável
loyalty_movements	PK id; FK user_id; FK pedido_id opcional	Entradas e débitos rastreáveis
audit	PK id; FK actor_id	Ação, recurso e instante; sem credenciais
idempotency	PK ator/operação/chave; FK actor_id	Fingerprint impede reuso da chave com outro conteúdo

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Feature crítica: realizar pedido e solicitar pagamento

Atores: Cliente (APP/WEB/TOTEM/PICKUP), Atendente (BALCAO), Cozinha, Gerente/Administrador e Gateway mock.

Pré-condições: usuário ativo autenticado; unidade ativa; produtos ativos, ofertados e disponíveis; cliente válido; chave Idempotency-Key; canal válido. Atendente/gerente informa clienteId e só atua na unidade vinculada. Cliente atua em seus próprios pedidos.

Fluxo principal:

1. A API valida canalPedido, itens e quantidades. O servidor calcula preços e descontos, sem confiar em valores monetários enviados pelo cliente.

2. Uma transação verifica estoque e consentimento/saldo para resgate; cria pedido e itens; registra saídas de estoque e eventual débito de pontos; grava auditoria e chave de idempotência.

3. Pedido inicia em AGUARDANDO_PAGAMENTO. O cliente solicita pagamento mock com chave de idempotência própria.

4. O gateway simulado responde com status e payload. A aplicação persiste a tentativa; apenas APROVADO altera o pedido para EM_PREPARO.

5. Cozinha ou atendimento marca PRONTO. Atendimento/gerente/admin marca ENTREGUE.

6. Na entrega, cliente com consentimento ativo recebe totalCentavos // 100 pontos; reentrega não pode creditar novamente.

Pós-condições: pedido e canal consultáveis, estoque atualizado, pagamento registrado e auditoria persistida. Saldo de pontos só aumenta após entrega.

Exceções: 401 sem autenticação; 403 perfil/unidade/cliente incompatível; 404 recurso inexistente; 422 canal ausente/inválido ou itens repetidos; 409 estoque/pontos insuficientes, cupom inválido, transição proibida ou chave reutilizada com corpo diferente. NEGADO e ERRO são resultados persistidos do gateway (HTTP 201 porque a tentativa foi criada), sem avanço do status. Nova tentativa usa nova chave. Reenvio com mesma chave/corpo devolve o recurso existente no estado atual, sem repetir efeitos.

Cancelamento antes da entrega devolve estoque exatamente uma vez; pagamento aprovado recebe ESTORNADO mock. Cliente pode cancelar apenas enquanto aguarda pagamento; equipe de atendimento autorizada pode cancelar EM_PREPARO/PRONTO. Cozinha pode somente marcar PRONTO. Pedido entregue

não pode ser cancelado no MVP. Pontos resgatados voltam no cancelamento se o consentimento ainda estiver ativo. Revogação de consentimento zera o saldo e impede novos créditos e devoluções enquanto revogado; o benefício já aplicado ao pedido não é removido.

Pedidos não pagos mantêm reserva de estoque até cancelamento manual. Expiração automática, reembolso real, compra anônima em totem, login refresh/logout com revogação individual e campanhas dinâmicas são evoluções propostas; não foram implementadas.

4. API e contratos

Consulte docs/api.md e docs/openapi.json; na API em execução, /docs e /openapi.json. Cada endpoint tem parâmetros, contratos tipados, permissões descritas e exemplos. Operações administrativas usam cadastro de cliente seguido de atribuição de perfil pelo ADMIN. Não há elevação de perfil pelo cadastro público.

Padrão de erro: {error, message, details:[{field,issue}], timestamp, path, requestId}. Detalhes de validação omitem valores de entrada para não repetir senhas. Listagens retornam {items,page,limit}. Dinheiro usa centavos inteiros, evitando erro de ponto flutuante. IDs históricos são preservados; recursos são desativados com ativo=false, sem apagar vendas.

Status: AGUARDANDO_PAGAMENTO → EM_PREPARO (apenas pagamento aprovado) → PRONTO → ENTREGUE. CANCELADO é terminal. Não há salto direto de AGUARDANDO_PAGAMENTO para PRONTO/ENTREGUE.

5. LGPD, privacidade, segurança e auditoria

Dados coletados: nome, email e hash de senha para identificação/autenticação; identificador interno, itens, unidade, canal, preços e datas para executar e acompanhar pedidos; perfil e unidade de funcionários para autorização; consentimento (estado, versão 1.0, instante) e movimentos de pontos para fidelização. Não são coletados CPF, telefone, endereço residencial ou dados de cartão. O endereço da unidade é comercial/fictício no seed.

Finalidade e bases legais propostas: execução de contrato para pedidos e conta; consentimento específico facultativo para fidelidade; logs estritamente necessários à segurança devem ter base e retenção revisadas pelo controlador. A descrição técnica não constitui parecer jurídico. A fidelidade é opcional e consentimento pode ser retirado por PUT /fidelidade/consentimento. O saldo é zerado em retirada, com movimento de histórico e evento de auditoria. A versão e o instante do consentimento/revogação são persistidos; o histórico de alterações consta na auditoria.

Senhas: BCrypt com salt aleatório por usuário (custo 12); comparação em tempo constante; jamais retornadas. JWT: HS256, sub/iat/exp/iss/aud obrigatórios, expiração padrão de 3600 segundos e segredo aleatório local de 256 bits. O perfil ativo é consultado no banco a cada requisição, portanto alteração de perfil e desativação se aplicam a tokens já emitidos. A chave privada .env tem permissões 0600 e está ignorada pelo Git.

Autorização: clientes acessam somente recursos próprios; empregados só recursos da unidade atribuída; ADMIN administra a rede. Logs de leitura de perfil/pedidos/pagamentos/fidelidade/auditoria e eventos de criação/status/estoque/consentimento usam apenas IDs e ação, sem credenciais, email, corpo de requisição ou dados de cartão. O endpoint de auditoria é exclusivo do ADMIN.

DELETE /usuarios/me exige perfil CLIENTE e nenhum pedido em aberto. Nome/email/hash são substituídos, consentimento retirado, saldo zerado e conta desativada; tokens antigos deixam de funcionar. IDs e histórico operacional permanecem: trata-se de desidentificação/pseudonimização local, e não garantia de anonimização irreversível em sistemas externos.

Estratégia de retenção proposta: revisar periodicamente contas inativas e necessidade de dados; propor 180 dias para auditoria de desenvolvimento, ajustados à finalidade e obrigações aplicáveis; manter registros de venda pelo prazo definido pelo controlador. Limpeza por prazo não é automática no MVP. Exportação pessoal, gestão de solicitações LGPD e descarte de backups exigem processo operacional futuro.

Antes de produção: definir política de retenção aplicável, HTTPS, armazenamento central de segredos, proteção contra força bruta/rate limiting, controle de backup/restauração e segregação de dados. O presente ambiente é destinado ao estudo e não valida capacidade/segurança de operação pública.

6. Plano de testes e evidências

A coleção Postman possui 23 cenários ordenados em sete pastas, positivos e negativos, com testes de status e conteúdo. docs/plano-testes.md apresenta entrada, pré-condições e resultado esperado de cada cenário; docs/evidencias-colecao-java.json é o resumo sanitizado da execução real em Java via JUnit e HttpClient. A suíte JUnit 5 possui 33 testes em banco temporário, incluindo concorrência de estoque/idempotência, isolamento entre clientes/unidades, pagamento negado/indisponível, consentimento, anonimização, migração em banco persistente existente e repetição de seed. docs/evidencias-junit-api.xml, docs/evidencias-junit-colecao.xml e docs/evidencias-junit-migracao.xml registram a execução atual.

Os resultados são evidência da máquina atual e do código correspondente; novas alterações exigem nova execução. Importar a coleção não prova aprovação dos cenários. Contagem de testes, falhas e data real constam nos arquivos de evidência. Uma validação adicional com Newman 6.2.1 executou os scripts Postman reais: 23 requisições e 46 asserções aprovadas, sem falhas; resumo em docs/evidencias-postman-newman.json. Testes não publicam o ambiente nem validam restauração de snapshot em uma nova tarefa.

7. Entrega técnica

Os artefatos incluem código Java, migrations Flyway, scripts de JVM/Maven, pom.xml, seed, .env.example, README, OpenAPI, coleção, plano de testes e diagramas. O documento único incorpora contratos, cenários, evidências sanitizadas e diagramas nos apêndices/anexos. Repositório: https://github.com/jefersonrivaldo/raizes-do-nordeste . A referência da revisão técnica é a branch main no commit 0756613 como base anterior às correções de coleção e migrações desta revisão. O conteúdo testado teve 33 testes aprovados, sem falhas ou testes ignorados.

Procedimento de reprodução: instalar JDK 21 e Maven 3.9+, executar ./scripts/mvn.sh verify, gerar a configuração local com java scripts/InitLocal.java e iniciar ./scripts/start.sh. A aplicação aplica Flyway e seed automaticamente; não é necessário criar um serviço H2 separado. O banco persistente fica em data/raizes.mv.db. Variáveis privadas são descritas em .env.example; o arquivo .env não deve ser publicado.

Swagger local: http://localhost:8000/docs ; contrato OpenAPI: http://localhost:8000/openapi.json ; coleção: https://github.com/jefersonrivaldo/raizes-do-nordeste/blob/main/docs/postman_collection.json . As URLs locais são acessadas depois de iniciar a API na máquina do avaliador. Na coleção, definir baseUrl e adminPassword em ambiente privado, executar T01–T23 em ordem e usar o seed documentado. As instruções completas estão no README.

Situação de acesso: repositório e coleção retornaram HTTP 200 sem autenticação na revisão de 11 out. 2026. O acesso público está disponível para avaliação. O ambiente local foi validado; não foi realizado deploy público. A apresentação de uma URL no relatório não comprova que ela esteja acessível ao avaliador.

8. Conclusão

O fluxo crítico é implementado com persistência real e operações transacionais. As limitações são explicitadas para separar capacidades demonstradas de evoluções propostas. Para entrega acadêmica, o estudante deve revisar o código/relatório, conferir seus dados de identificação, verificar a versão oficial do roteiro, verificar a visibilidade do repositório e testar os links pelo acesso do avaliador. Histórico mínimo de cinco commits é uma recomendação do roteiro; não foi fabricado nesta implementação.

REFERÊNCIAS

UNINTER. Projeto Multidisciplinar: orientações e estudo de caso — rede Raízes do Nordeste. Versão editada para estudo, 2026. Arquivo fornecido pelo estudante: UNINTER_Estudo_de_Caso_Versao_Editada_Estudo.pdf. Contém omissões; o documento oficial prevalece.

UNINTER. Projeto Multidisciplinar: trilha Back-End — rede Raízes do Nordeste. Versão editada para estudo, 2026. Arquivo fornecido pelo estudante: UNINTER_Trilha_Back_End_Versao_Editada_Estudo.pdf. Contém omissões; o documento oficial prevalece.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018 (LGPD). https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm . Referência normativa indicada, sem alegar consulta jurídica.

SPRING. Documentação oficial do Spring Boot. https://docs.spring.io/spring-boot/ . Referência técnica para aprofundamento.

H2. Documentação do banco H2. https://h2database.com/html/main.html . Referência técnica para aprofundamento.

OWASP. Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html . Referência para revisão de configuração em produção.

APÊNDICE A — CONTRATOS E EXEMPLOS DA API

Os exemplos são ilustrativos e seguem os esquemas da API implementada. IDs e timestamps devem ser substituídos pelos dados da execução. Valores monetários são inteiros em centavos. Nem todo código de erro listado se aplica a toda operação.

Gerado do OpenAPI real do Spring Boot. Exemplos são ilustrativos. Valores em centavos; listagens retornam items/page/limit (limit até 100). Idempotency-Key tem 8–100 caracteres alfanuméricos, ponto, hífen ou sublinhado. Equipe informa clienteId na criação de pedido. Reenvio idempotente retorna estado atual.

PUT /usuarios/{id}/perfil

Atribuir perfil e unidade (ADMIN)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 4 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"perfil" : "CLIENTE",
"unidadeId" : 1
}
```

Response HTTP 200:

```
{
"id" : 1,
"nome" : "texto",
"perfil" : "CLIENTE",
"unidadeId" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

PUT /unidades/{id}

Atualizar unidade (ADMIN ou GERENTE da unidade)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 5 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"nome" : "texto",
"endereco" : "texto",
"ativo" : true
}
```

Response HTTP 200:

```
{
"id" : 1,
"nome" : "texto",
"endereco" : "texto",
"active" : true
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

PUT /produtos/{id}

Atualizar produto global (ADMIN)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 6 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"nome" : "texto",
"descricao" : "texto",
"ativo" : true
}
```

Response HTTP 200:

```
{
"id" : 1,
"nome" : "texto",
"descricao" : "texto",
"active" : true
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

PUT /fidelidade/consentimento

Registrar ou revogar consentimento próprio (CLIENTE)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

```
{
"consentimento" : true
}
```

Response HTTP 200:

```
{
"consentimento" : true,
"versao" : "texto"
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /usuarios

Cadastrar cliente (público; não aceita perfil)

Autenticação: público.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

```
{
"email" : "maria@example.com",
"nome" : "Maria de estudo",
"senha" : "SenhaEstudo123!"
}
```

Response HTTP 201:

```
{
"id" : 1,
"nome" : "texto",
"perfil" : "CLIENTE",
"unidadeId" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /unidades

Listar unidades (público)

Autenticação: público.

Parâmetros:

Tabela 7 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"items" : [ {
```

```
"id" : 1,
"nome" : "texto",
"endereco" : "texto",
"active" : true
} ],
"page" : 1,
"limit" : 1
}
```


Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /unidades

Cadastrar unidade (ADMIN)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:


```
{
"nome" : "texto",
"endereco" : "texto",
"ativo" : true
}
```


Response HTTP 201:


```
{
"id" : 1,
"nome" : "texto",
"endereco" : "texto",
"active" : true
}
```


Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /unidades/{id}/estoque/movimentos

Movimentar estoque e definir preço (ADMIN ou GERENTE da unidade)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 8 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"produtoId" : 1,
"quantidade" : 1,
"precoCentavos" : 1,
"motivo" : "texto"
}
```

Response HTTP 201:

```
{
"produtoId" : 1,
"quantidade" : 1,
"precoCentavos" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /produtos

Listar catálogo (público)

Autenticação: público.

Parâmetros:

Tabela 9 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"items" : [ {
"id" : 1,
"nome" : "texto",
"descricao" : "texto",
"active" : true
} ],
"page" : 1,
"limit" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /produtos

Cadastrar produto global (ADMIN)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

```
{
"nome" : "texto",
"descricao" : "texto",
"ativo" : true
}
```

Response HTTP 201:

```
{
"id" : 1,
"nome" : "texto",
"descricao" : "texto",
"active" : true
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /pedidos

Consultar pedidos autorizados com filtro por canal/status (JWT)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 10 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}
canalPedido	query	false	{"type":"string","enum":["APP","TOTEM","BALCAO","PICKUP","WEB"]}
status	query	false	{"type":"string","enum":["AGUARDANDO_PAGAMENTO","EM_PREPARO","PRONTO","ENTREGUE","CANCELADO"]}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:


```
{
"items" : [ {
"id" : 1,
"clienteId" : 1,
"unidadeId" : 1,
"canalPedido" : "APP",
"status" : "AGUARDANDO_PAGAMENTO",
"subtotalCentavos" : 1,
"descontoCentavos" : 1,
"totalCentavos" : 1,
"pontosResgate" : 1,
"createdAt" : "texto",
"itens" : [ {
"produtoId" : 1,
"nome" : "texto",
"quantidade" : 1,
"precoUnitarioCentavos" : 1
} ]
} ],
"page" : 1,
"limit" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /pedidos

Criar pedido multicanal (cliente próprio ou atendimento na unidade)

Idempotency-Key obrigatório. Equipe informa clienteId. Estoque e pontos são debitados atomicamente; reenvio da mesma chave/corpo devolve o recurso atual.

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 11 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
Idempotency-Key	header	true	{"type":"string","maxLength":100,"minLength":8,"pattern":"[A-Za-z0-9_.-]+"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"formaPagamento" : "MOCK",
"itens" : [ {
"quantidade" : 2,
"produtoId" : 1
} ],
"canalPedido" : "TOTEM",
"unidadeId" : 1
}
```

Response HTTP 201:

```
{
"id" : 1,
"clienteId" : 1,
"unidadeId" : 1,
"canalPedido" : "APP",
"status" : "AGUARDANDO_PAGAMENTO",
"subtotalCentavos" : 1,
"descontoCentavos" : 1,
"totalCentavos" : 1,
"pontosResgate" : 1,
"createdAt" : "texto",
"itens" : [ {
"produtoId" : 1,
"nome" : "texto",
"quantidade" : 1,
"precoUnitarioCentavos" : 1
```

```
} ]
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /pedidos/{id}/pagamentos

Listar pagamentos próprios ou da unidade autorizada (JWT)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 12 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"items" : [ {
"id" : 1,
"pedidoId" : 1,
"status" : "texto",
"valorCentavos" : 1,
"payload" : { }
} ],
"page" : 1,
"limit" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /pedidos/{id}/pagamentos

Registrar tentativa de pagamento MOCK (cliente próprio ou atendimento autorizado)

HTTP 201 significa tentativa criada. Examine APROVADO/NEGADO/ERRO no corpo; nova tentativa exige nova chave. Apenas aprovação permite EM_PREPARO.

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 13 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}
Idempotency-Key	header	true	{"type":"string","maxLength":100,"minLength":8,"pattern":"[A-Za-z0-9_.-]+"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"cenario" : "APROVAR"
}
```

Response HTTP 201:

```
{
"id" : 1,
"pedidoId" : 1,
"status" : "texto",
"valorCentavos" : 1,
"payload" : { }
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

POST /auth/login

Login público e emissão de JWT

Autenticação: público.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

```
{
"senha" : "SenhaEstudo123!",
"email" : "maria@example.com"
}
```

Response HTTP 200:

```
{
"accessToken" : "texto",
"tokenType" : "texto",
"expiresIn" : 1,
"user" : {
"id" : 1,
"nome" : "texto",
"perfil" : "CLIENTE",
"unidadeId" : 1
}
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

PATCH /pedidos/{id}/status

Avançar status ou cancelar conforme perfil e unidade

Cliente cancela apenas antes de pagar; cozinha apenas marca PRONTO; atendimento avança PRONTO/ENTREGUE ou cancela antes da entrega.

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 14 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

```
{
"status" : "AGUARDANDO_PAGAMENTO"
}
```

Response HTTP 200:

```
{
"id" : 1,
```

```
"clienteId" : 1,
"unidadeId" : 1,
"canalPedido" : "APP",
"status" : "AGUARDANDO_PAGAMENTO",
"subtotalCentavos" : 1,
"descontoCentavos" : 1,
"totalCentavos" : 1,
"pontosResgate" : 1,
"createdAt" : "texto",
"itens" : [ {
"produtoId" : 1,
"nome" : "texto",
"quantidade" : 1,
"precoUnitarioCentavos" : 1
} ]
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /usuarios/me

Consultar perfil próprio (JWT)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

Sem corpo.

Response HTTP 200:

```
{
"id" : 1,
"nome" : "texto",
"perfil" : "CLIENTE",
"unidadeId" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

DELETE /usuarios/me

Desidentificar conta própria (CLIENTE, sem pedidos abertos)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

Sem corpo.

Response HTTP 204:

Sem corpo.

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /unidades/{id}/estoque

Consultar estoque (ADMIN ou GERENTE da unidade)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 15 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:


```
{
"items" : [ {
"produtoId" : 1,
"quantidade" : 1,
"precoCentavos" : 1
} ],
"page" : 1,
"limit" : 1
```

```
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /unidades/{id}/cardapio

Consultar cardápio por unidade (público)

Autenticação: público.

Parâmetros:

Tabela 16 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"items" : [ {
"produtoId" : 1,
"nome" : "texto",
"descricao" : "texto",
"precoCentavos" : 1,
"disponivel" : 1
} ],
"page" : 1,
"limit" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /pedidos/{id}

Consultar pedido próprio ou da unidade autorizada (JWT)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 17 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
id	path	true	{"type":"integer","format":"int64"}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"id" : 1,
"clienteId" : 1,
"unidadeId" : 1,
"canalPedido" : "APP",
"status" : "AGUARDANDO_PAGAMENTO",
"subtotalCentavos" : 1,
"descontoCentavos" : 1,
"totalCentavos" : 1,
"pontosResgate" : 1,
"createdAt" : "texto",
"itens" : [ {
"produtoId" : 1,
"nome" : "texto",
"quantidade" : 1,
"precoUnitarioCentavos" : 1
} ]
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /health

Verificar API e banco (público)

Autenticação: público.

Parâmetros:

Não se aplicam parâmetros path/query nesta operação.

Request:

Sem corpo.

Response HTTP 200:

```
{ }
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /fidelidade

Consultar saldo e histórico próprio (CLIENTE)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 18 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"saldo" : 1,
"consentimento" : true,
"consentimentoEm" : "texto",
"versao" : "texto",
"items" : [ {
"pedidoId" : 1,
"quantidade" : 1,
"motivo" : "texto",
"createdAt" : "texto"
} ],
"page" : 1,
"limit" : 1
```

```
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

GET /auditoria

Consultar auditoria (ADMIN)

Autenticação: JWT Bearer; perfil/unidade conforme resumo.

Parâmetros:

Tabela 19 — Nome e informações relacionadas

Nome	Local	Obrigatório	Contrato
page	query	false	{"type":"integer","format":"int32","default":1,"maximum":1000000,"minimum":1}
limit	query	false	{"type":"integer","format":"int32","default":20,"maximum":100,"minimum":1}

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Request:

Sem corpo.

Response HTTP 200:

```
{
"items" : [ {
"id" : 1,
"actor_id" : 1,
"acao" : "texto",
"recurso" : "texto",
"created_at" : "texto"
} ],
"page" : 1,
"limit" : 1
}
```

Erros: 401 autenticação, 403 autorização, 404 inexistente, 409 regra/integridade, 422 validação, 503 banco ocupado; JSON ErrorResponse uniforme. Nem todos os erros se aplicam a toda rota.

Padrão de erro

```
{"error":"ESTOQUE_INSUFICIENTE","message":"Estoque insuficiente na
unidade.","details":[],"timestamp":"2026-10-11T15:00:00Z","path":"/pedidos","requ
estId":"uuid"}
```

HTTP 201 em pagamento significa tentativa persistida; examine status APROVADO/NEGADO/ERRO. Erros de validação omitem os valores enviados.

APÊNDICE B — CENÁRIOS DE TESTE DA API

Executar no Postman as sete pastas em ordem, mantendo T01–T23; a mesma coleção é validada pelo teste Java CollectionContractTest. Executar na ordem indicada com banco criado pelo seed. Cada cenário tem asserções na coleção. CLIENTE é criado em cada execução; adminPassword é configurado localmente e não deve ser exportado. Unidade 1 e produto 1 vêm do seed.

Tabela 20 — ID e informações relacionadas

ID	Requisição e entrada	Pré-condição	Saída esperada
T01 Cadastro de cliente	POST /usuarios; {"nome": "Cliente de teste", "email": "{{clientEmail}}", "senha": "{{clientPassword}}"}	API iniciada; email novo gerado automaticamente	HTTP 201; asserções na requisição homônima
T02 Login cliente	POST /auth/login; {"email": "{{clientEmail}}", "senha": "{{clientPassword}}"}	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima
T03 Login administrador	POST /auth/login; {"email": "admin@raizes.local", "senha": "{{adminPassword}}"}	Seed aplicado; variável adminPassword informada localmente	HTTP 200; asserções na requisição homônima
T04 Cardápio por unidade	GET /unidades/1/cardapio; Sem corpo	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima
T05 Consentimento fidelidade	PUT /fidelidade/consentimento; {"consentimento": true}	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima
T06 Criar pedido multicanal	POST /pedidos; {"unidadeId": 1, "canalPedido": "TOTEM", "itens": [{"produtoId": 1, "quantidade": 2}], "formaPagamento": "MOCK"}	Fluxo anterior concluído	HTTP 201; asserções na requisição homônima
T07 Idempotência pedido	POST /pedidos; {"unidadeId": 1, "canalPedido": "TOTEM", "itens": [{"produtoId": 1, "quantidade": 2}], "formaPagamento": "MOCK"}	Fluxo anterior concluído	HTTP 201; asserções na requisição homônima
T08 Pagamento negado	POST /pedidos/{{pedidoId}}/pagamentos; {"cenario": "NEGAR"}	Fluxo anterior concluído	HTTP 201; asserções na requisição homônima
T09 Pagamento aprovado em nova tentativa	POST /pedidos/{{pedidoId}}/pagamentos; {"cenario": "APROVAR"}	Fluxo anterior concluído	HTTP 201; asserções na requisição homônima
T10 Cozinha concluiu preparo	PATCH /pedidos/{{pedidoId}}/status; {"status": "PRONTO"}	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima
T11 Entrega e fidelização	PATCH /pedidos/{{pedidoId}}/status; {"status": "ENTREGUE"}	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima
T12 Saldo fidelidade	GET /fidelidade; Sem corpo	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima
T13 Filtro canalPedido	GET /pedidos?canalPedido=TOTEM&status=ENTREGUE&page=1&limit=20; Sem corpo	Fluxo anterior concluído	HTTP 200; asserções na requisição homônima

ID	Requisição e entrada	Pré-condição	Saída esperada
T14 Sem autenticação	GET /pedidos; Sem corpo	Omitir Authorization	HTTP 401; asserções na requisição homônima
T15 Perfil sem permissão	POST /unidades; {"nome": "Unidade teste", "endereco": "Rua teste, 123"}	Fluxo anterior concluído	HTTP 403; asserções na requisição homônima
T16 Canal obrigatório ausente	POST /pedidos; {"unidadeId": 1, "itens": [{"produtoId": 1, "quantidade": 2}], "formaPagamento": "MOCK"}	Fluxo anterior concluído	HTTP 422; asserções na requisição homônima
T17 Canal inválido	POST /pedidos; {"unidadeId": 1, "canalPedido": "OUTRO", "itens": [{"produtoId": 1, "quantidade": 2}], "formaPagamento": "MOCK"}	Fluxo anterior concluído	HTTP 422; asserções na requisição homônima
T18 Estoque insuficiente	POST /pedidos; {"unidadeId": 1, "canalPedido": "TOTEM", "itens": [{"produtoId": 1, "quantidade": 1000}], "formaPagamento": "MOCK"}	Seed com saldo de 100; não adicionar estoque antes deste teste	HTTP 409; asserções na requisição homônima
T19 Pedido inexistente	GET /pedidos/999999999; Sem corpo	Fluxo anterior concluído	HTTP 404; asserções na requisição homônima
T20 Transição inválida	PATCH /pedidos/{{pedidoId}}/status; {"status": "PRONTO"}	Fluxo anterior concluído	HTTP 409; asserções na requisição homônima

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Tabela 21 — T21 Unidade inexistente e informações relacionadas

T21 Unidade inexistente	POST /pedidos; unidadeId = 999999999; demais campos válidos	CLIENTE autenticado; chave de idempotência nova	HTTP 404; envelope de erro padronizado
T22 Produto inexistente	POST /pedidos; produtoId = 999999999; demais campos válidos	CLIENTE autenticado; chave de idempotência nova	HTTP 404; envelope de erro padronizado
T23 Auditoria de criação do pedido	GET /auditoria?page=1&limit=100	ADMIN autenticado; pedido criado anteriormente	HTTP 200; items contém evento PEDIDO_CRIADO

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Critérios de aceitação e evidências por cenário

A evidência de cada linha é a requisição de mesmo ID e nome em docs/postman_collection.json. T01–T13 validam a sequência operacional e T14–T20 validam erros; T08 é uma negativa de negócio do gateway, com HTTP 201 por registrar a tentativa. As asserções da coleção validam o conteúdo abaixo, além do status HTTP.

Tabela 22 — ID e informações relacionadas

ID	Conteúdo esperado da resposta	Evidência e uso
T01	perfil = CLIENTE	Requisição T01; cadastro sem elevação de perfil
T02	accessToken é uma string	Requisição T02; grava token para as próximas chamadas
T03	user.perfil = ADMIN	Requisição T03; grava adminToken em variável local
T04	items[0].produtoId = 1; precoCentavos = 2500	Requisição T04; depende dos dados fictícios do seed
T05	consentimento = true	Requisição T05; fidelidade facultativa habilitada
T06	status = AGUARDANDO_PAGAMENTO; totalCentavos = 5000; canalPedido = TOTEM	Requisição T06; grava pedidoId
T07	id igual ao pedidoId de T06	Requisição T07; mesma chave e corpo, sem novo pedido
T08	status = NEGADO	Requisição T08; tentativa recusada registrada
T09	status = APROVADO	Requisição T09; nova chave autoriza a progressão do pedido
T10	status = PRONTO	Requisição T10; usa token administrativo autorizado
T11	status = ENTREGUE	Requisição T11; entrega finaliza o fluxo
T12	saldo = 50	Requisição T12; 50 reais pagos correspondem a 50 pontos
T13	items contém pedidoId com canalPedido = TOTEM	Requisição T13; filtro por canal e estado entregue
T14	error, message, details, timestamp, path e requestId	Requisição T14; erro 401 sem Authorization
T15	Mesmas seis chaves do padrão de erro	Requisição T15; erro 403 usando token CLIENTE
T16	Mesmas seis chaves do padrão de erro	Requisição T16; erro 422 por ausência do canal
T17	Mesmas seis chaves do padrão de erro	Requisição T17; erro 422 para canal OUTRO
T18	Mesmas seis chaves do padrão de erro	Requisição T18; erro 409 por quantidade superior ao saldo
T19	Mesmas seis chaves do padrão de erro	Requisição T19; erro 404 para pedido inexistente
T20	Mesmas seis chaves do padrão de erro	Requisição T20; erro 409 ao tentar reabrir pedido entregue

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Tabela 23 — T21 e informações relacionadas

T21	Mesmas seis chaves do padrão de erro	Requisição T21; unidade inexistente na criação de pedido
T22	Mesmas seis chaves do padrão de erro	Requisição T22; produto inexistente na criação de pedido
T23	items contém evento PEDIDO_CRIADO	Requisição T23; consulta ADMIN comprova auditoria

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Testes automatizados complementares

Tabela 24 — Teste JUnit e informações relacionadas

Teste JUnit	Entrada e pré-condição	Resultado verificado
auditoriaEOpenApi	Cliente cria pedido; ADMIN consulta GET /auditoria	HTTP 200; registro PEDIDO_CRIADO presente; canalPedido obrigatório no esquema
cupomERecursoAusente	Pedido com NORDESTE10; depois unidadeId = 999	Desconto produz totalCentavos = 4500; unidade ausente retorna 404
quantidadeFracionariaRejeitada	POST /pedidos com quantidade = 1.5	HTTP 422; inteiro exigido

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Esses três casos são testes Java, não requisições adicionais da coleção Postman. A coleção contém 23 cenários em sete pastas, incluindo auditoria e unidade/produto inexistente. A suíte Java verifica comportamentos adicionais sem substituir os artefatos solicitados pela trilha.

APÊNDICE C — EVIDÊNCIAS DE EXECUÇÃO

Os registros abaixo provêm dos artefatos de execução disponíveis no projeto. A execução de teste em banco temporário é distinta da verificação de persistência em arquivo após reinício. Não são utilizados dados pessoais de clientes reais.

Tabela 25 — Suíte e informações relacionadas

Suíte	Executados	Falhas / erros / ignorados
ApiIntegrationTest	31	0 / 0 / 0
CollectionContractTest	1	0 / 0 / 0
DatabaseMigrationTest	1	0 / 0 / 0

Fonte: artefatos do projeto Raízes do Nordeste (2026).

Comando de validação: mvn verify. A execução registrada nesta revisão de 11 out. 2026 aprovou 33 testes Java: 31 de integração, um executor de 23 requisições da coleção e um de migração em arquivo H2 existente; zero falhas, erros ou ignorados. O executor percorre as sete pastas da coleção na ordem do fluxo. Os relatórios sanitizados estão em docs/evidencias-junit-api.xml, docs/evidencias-junit-colecao.xml e docs/evidencias-junit-migracao.xml.

Registro: docs/evidencias-colecao-java.json. A data executadoEm está expressa em UTC. A coleção foi validada em Java com asserções equivalentes em Java; o arquivo separado de Newman registra a execução dos scripts Postman reais. O registro de runtime mantém sua data histórica.


```
{
"executadoEm" : "2026-10-11T21:16:09.227056362Z",
"falhas" : 0,
"executor" : "Java 21 / JUnit 5 / HttpClient",
"requisicoes" : 23,
"gruposDeAssercoes" : 46
}
```


Registro: docs/evidencias-runtime-java.json. A data executadoEm está expressa em UTC. A coleção foi validada em Java com asserções equivalentes em Java; o arquivo separado de Newman registra a execução dos scripts Postman reais. O registro de runtime mantém sua data histórica.


```
{
"health" : true,
"pedidoId" : 1,
"openapiCompatível" : true,
"status" : "ENTREGUE",
"executadoEm" : "2026-10-11T15:25:17.272459562Z",
"tecnologia" : "Java 21 / Spring Boot / H2 / Flyway",
```

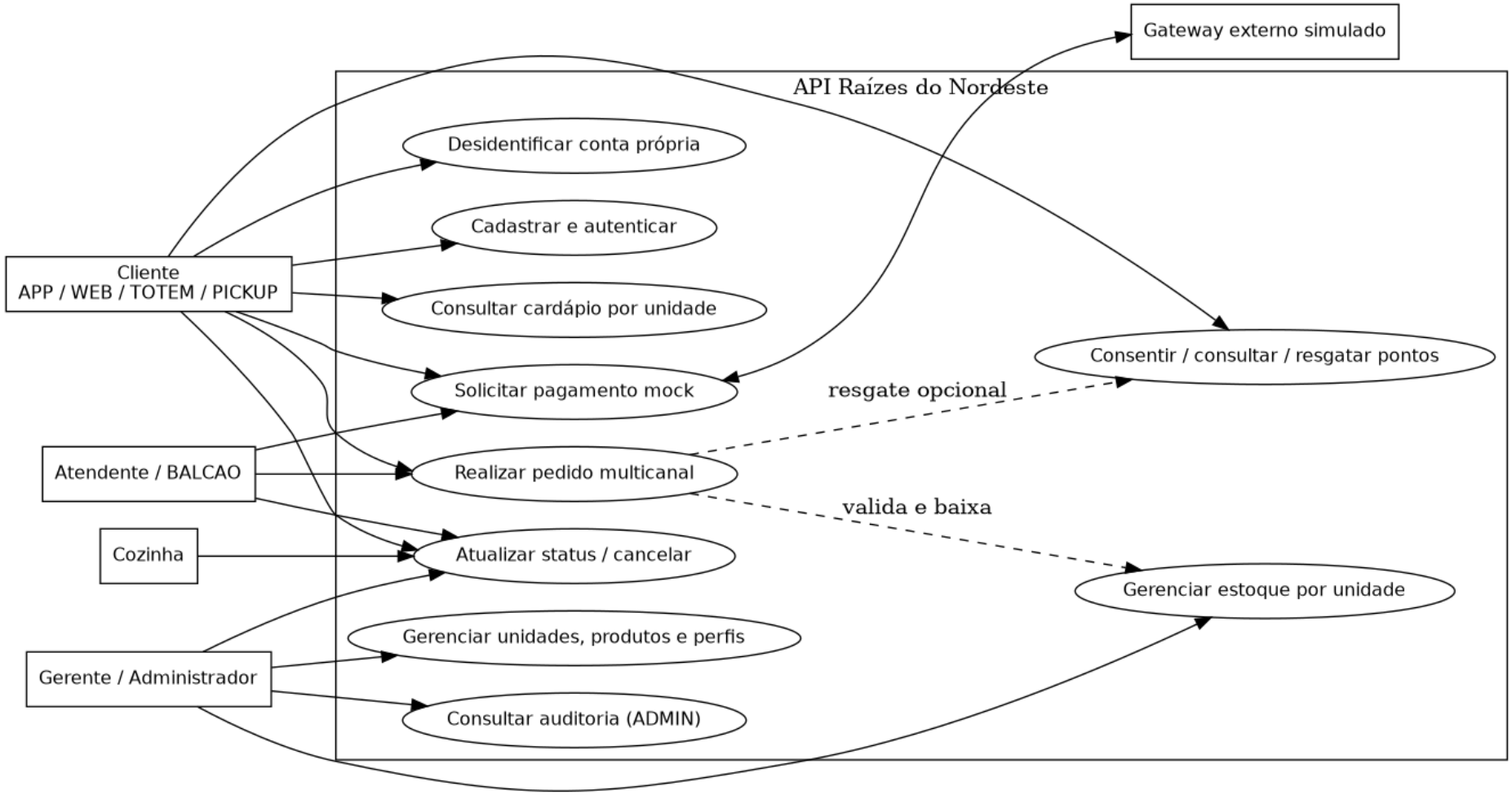
```
"swagger" : true,
"persistenciaAposReinicio" : true
}
```


Registro: docs/evidencias-postman-newman.json. A data executadoEm está expressa em UTC. A coleção foi validada em Java com asserções equivalentes em Java; o arquivo separado de Newman registra a execução dos scripts Postman reais. O registro de runtime mantém sua data histórica.


```
{
"executadoEm" : "2026-10-11T21:17:20.719407+00:00",
"executor" : "Newman 6.2.1",
"baseDaRevisao" : "07566134fbda88e93923c58b6dd407f999ba24ba",
"requisicoes" : 23,
"pastas" : 7,
"assercoes" : 46,
"falhas" : 0,
"positivos2xx" : 14,
"negativos4xx" : 9,
"banco" : "H2 em arquivo temporário isolado; removido após execução",
"scriptsPostmanExecutados" : true,
"credenciaisOmitidas" : true
}
```

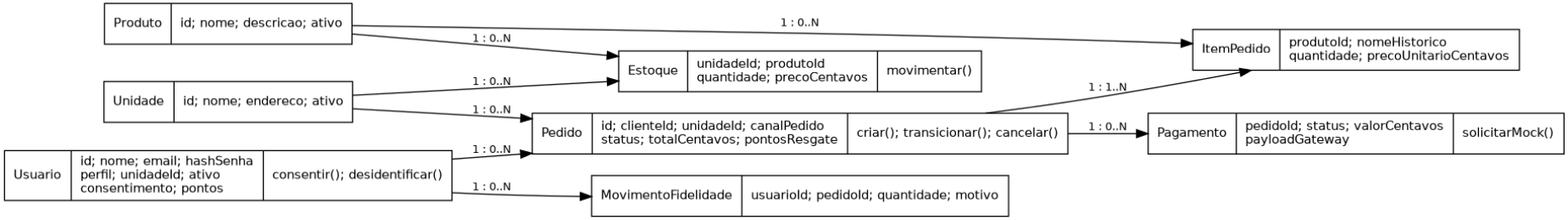

Cobertura complementar: auditoriaEOpenApi verifica PEDIDO_CRIADO na auditoria; cupomERecursoAusente verifica unidade inexistente (404); quantidadeFracionariaRejeitada verifica formato numérico inválido (422). Estes são testes JUnit adicionais, ainda não representados por requisições próprias na coleção Postman.

APÊNDICE D — DIAGRAMA DE CASOS DE USO



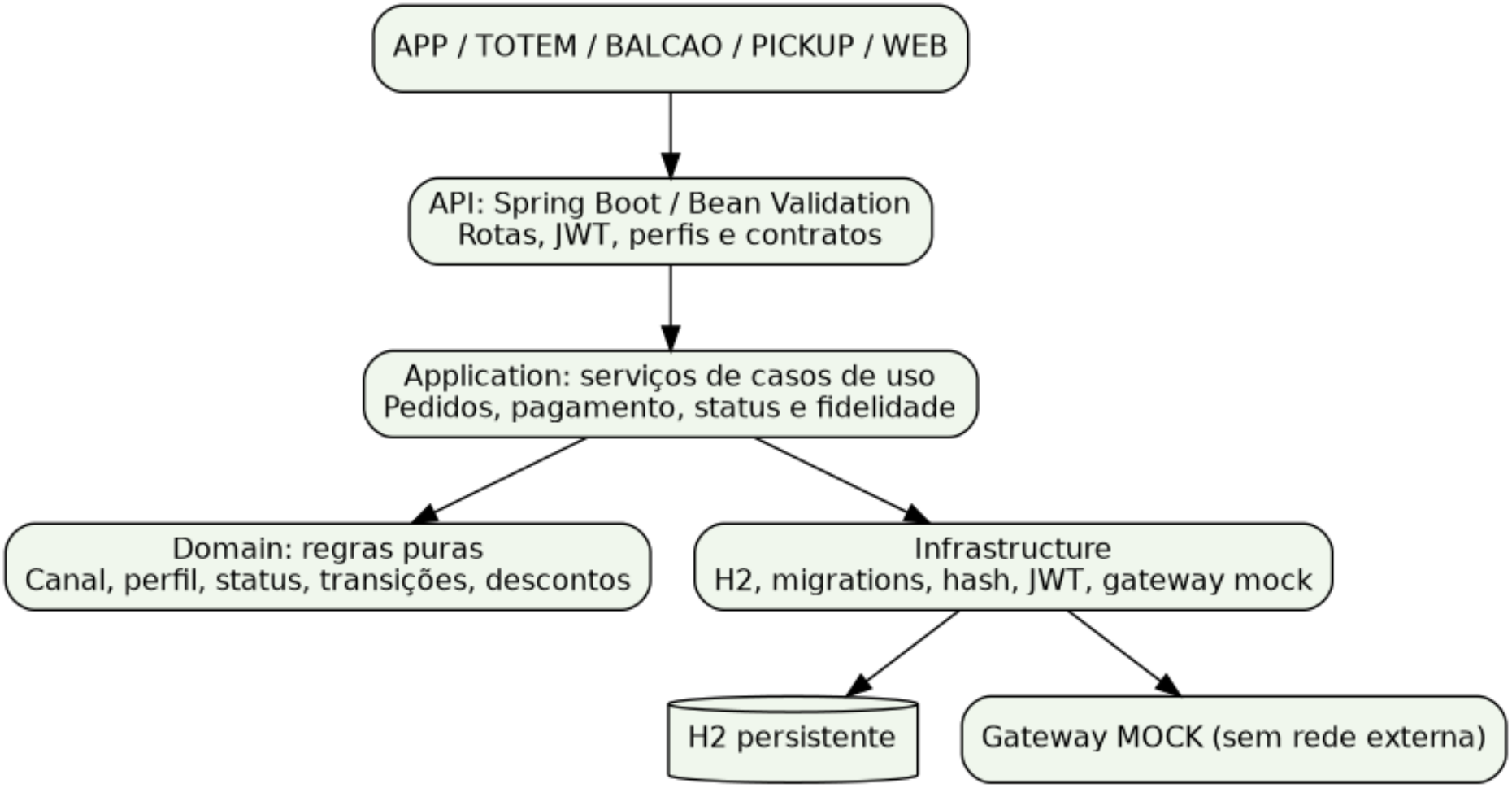
Fonte: elaboração para o projeto Raízes do Nordeste (2026).
Fonte editável e imagem ampliável: docs/diagramas/casos-de-uso.

APÊNDICE E — DIAGRAMA CONCEITUAL DE CLASSES



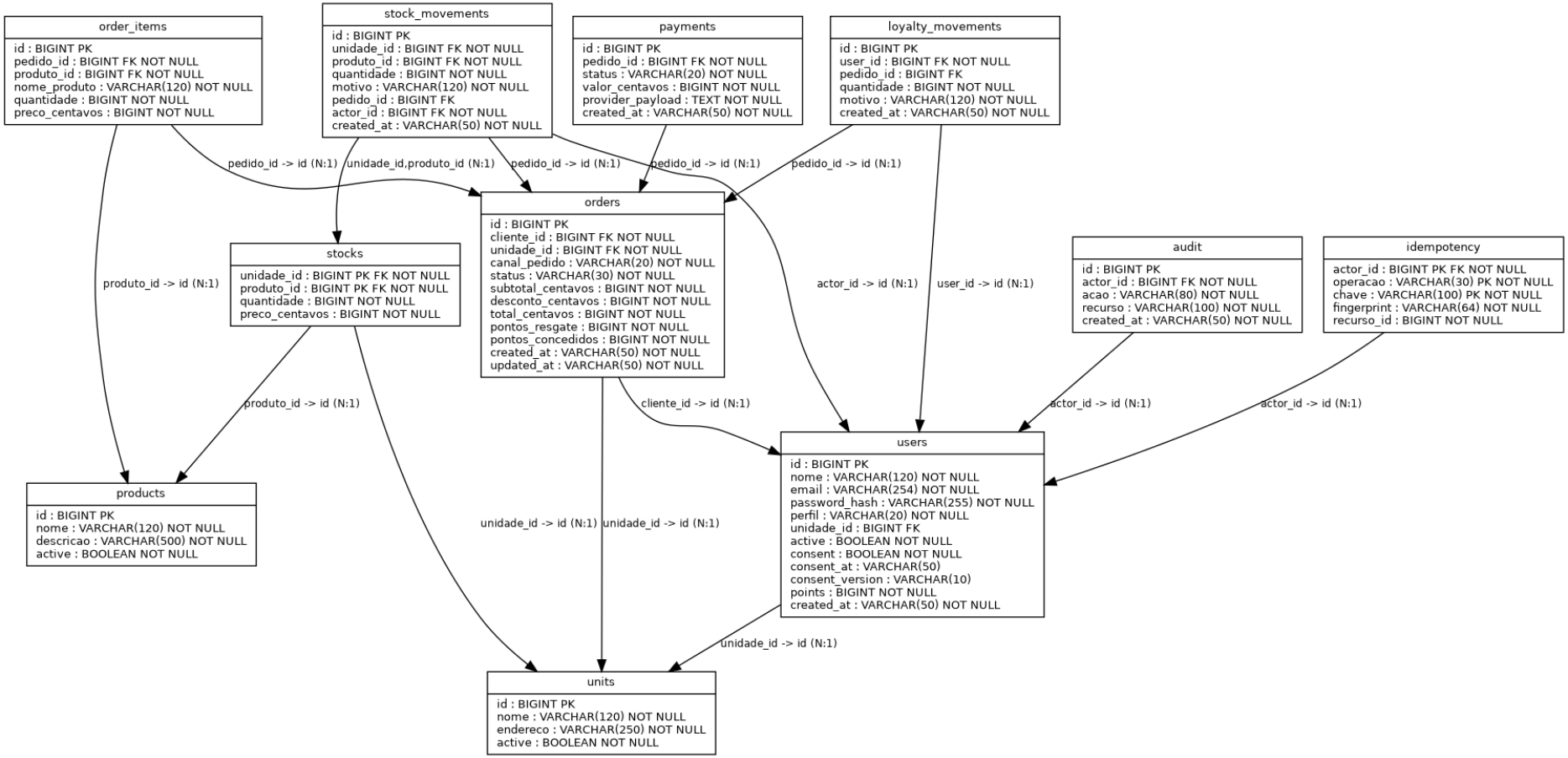
Fonte: elaboração para o projeto Raízes do Nordeste (2026).
Fonte editável e imagem ampliável: docs/diagramas/classes.

APÊNDICE F — ARQUITETURA EM CAMADAS



Fonte: elaboração para o projeto Raízes do Nordeste (2026).
Fonte editável e imagem ampliável: docs/diagramas/arquitetura.

APÊNDICE G — DIAGRAMA ENTIDADE-RELACIONAMENTO



Fonte: elaboração para o projeto Raízes do Nordeste (2026).

Fonte editável e imagem ampliável: docs/diagramas/der.